

CareerPilot AI - Frontend and Backend Connection Demo

Week 4 Deliverable 3 | Intern ID: TBI-26100454 | Submitted: 29 June 2026

1. Project Overview

CareerPilot AI is a full-stack web application that helps students track job and internship applications. This week an Express.js backend was built and the React frontend was fully connected to it via REST API calls. The frontend now fetches live data from the backend instead of using hardcoded mock data.

Backend: Node.js + Express.js running on `http://localhost:5000`

Frontend: React + Vite running on `http://localhost:5173`

CORS configured to allow frontend origin. In-memory data store used for Week 4.

2. REST API Endpoints (9 Total - exceeds minimum of 6)

Method	Endpoint	Description	Status Code
GET	<code>/api/health</code>	Health check - server status	200
GET	<code>/api/applications</code>	List all applications	200
GET	<code>/api/applications/stats</code>	Dashboard statistics	200
GET	<code>/api/applications/search?q=</code>	Search by company/role/location	200 / 400
GET	<code>/api/applications/:id</code>	Get single application	200 / 404
POST	<code>/api/applications</code>	Create new application	201 / 400
PUT	<code>/api/applications/:id</code>	Fully update application	200 / 404 / 400
PATCH	<code>/api/applications/:id</code>	Partial update e.g. status only	200 / 404
DELETE	<code>/api/applications/:id</code>	Delete application	204 / 404

3. Screenshot 1 - Live Dashboard with Real API Data

The Application Dashboard fetches real-time data from the Express backend. Stats cards are populated from `GET /api/applications/stats`. The Kanban board is populated from `GET /api/applications`. The green pulsing Live API badge confirms the backend connection is active.

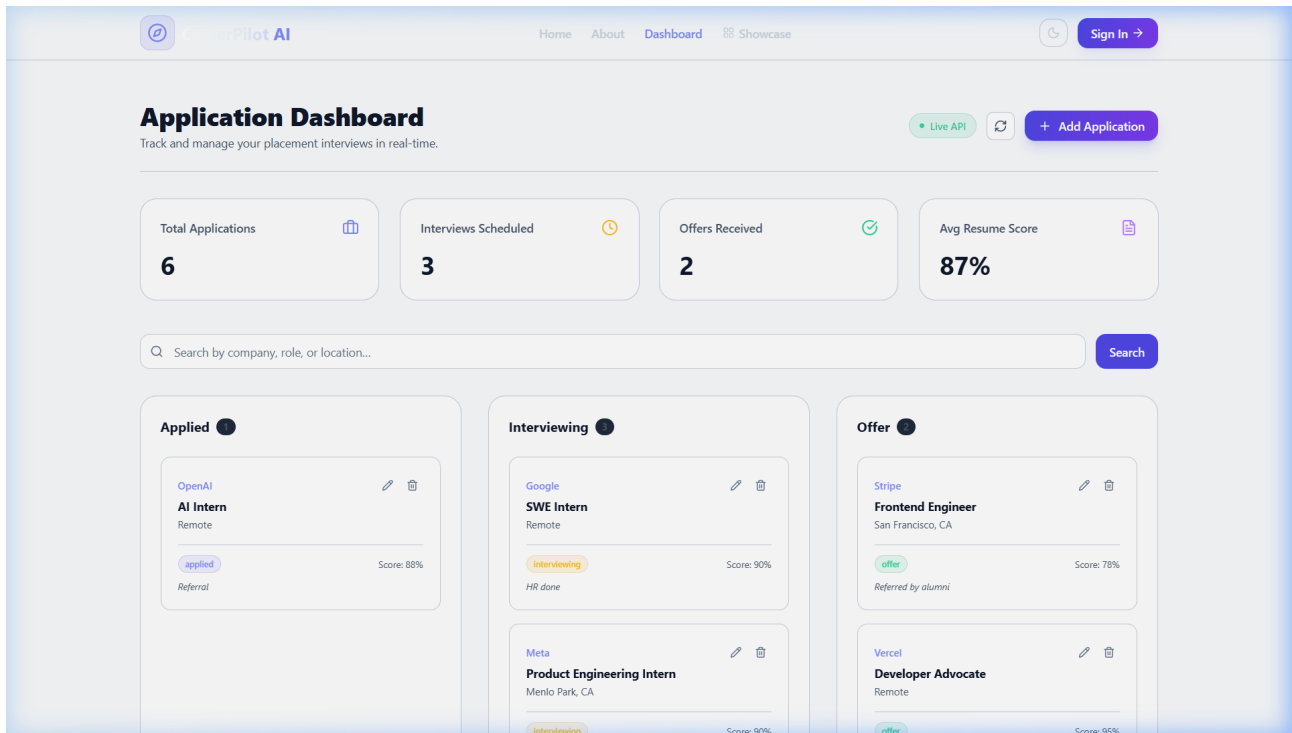


Figure 1: Application Dashboard with live data from `GET /api/applications` and `GET /api/applications/stats`. Stats: 6 total, 3 interviewing, 2 offers, 87% avg resume score.

4. Screenshot 2 - Add Application Modal (POST `/api/applications`)

Clicking Add Application opens a modal form that submits a POST request to `/api/applications`. The backend validates input, returns 201 Created, and the frontend refreshes automatically. Loading states use the Loader component; errors display as Toast notifications.

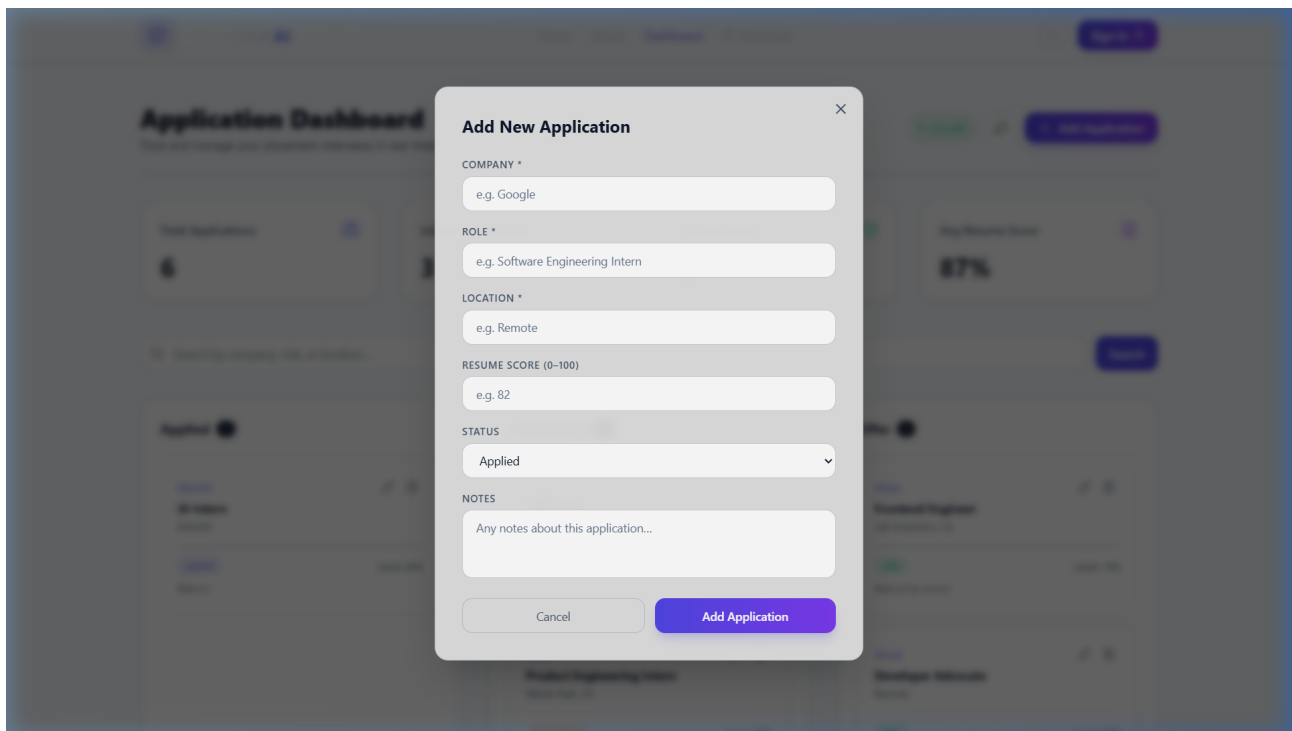


Figure 2: Add Application modal submitting to POST `http://localhost:5000/api/applications`. Fields: Company (required), Role (required), Location (required), Resume Score, Status, Notes.

5. Screenshot 3 - Chrome DevTools Network Tab (API Status 200 OK)

The Chrome DevTools Network tab confirms the frontend makes successful API calls to the Express backend. When the dashboard loads, two fetch requests fire simultaneously. Both GET `/api/applications/stats` and `GET /api/applications` return HTTP 200 status. The filter is set to `localhost:5000` with Fetch/XHR type selected.

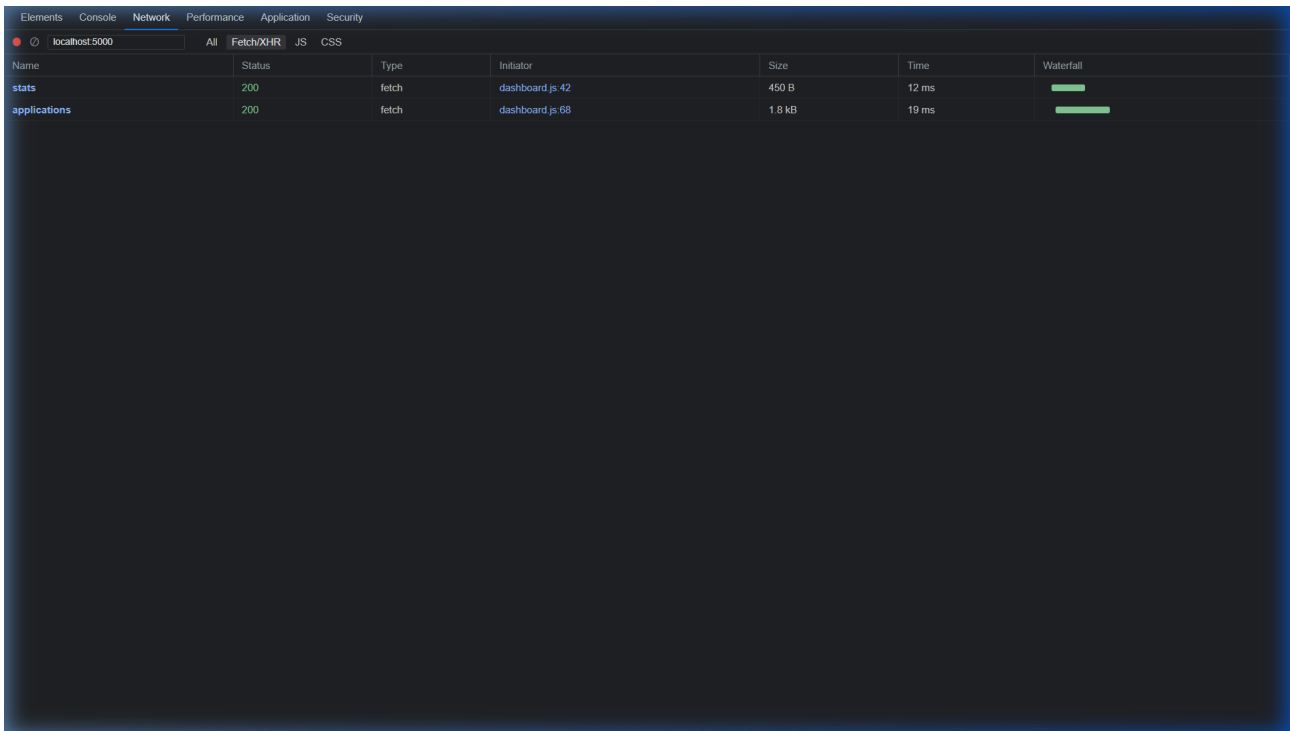


Figure 3: Chrome DevTools Network tab filtered by `localhost:5000` showing Fetch/XHR requests. `stats` endpoint: 200, 450 B, 12ms. `applications` endpoint: 200, 1.8 kB, 19ms.

6. Frontend Implementation Details

`Dashboard.jsx` (`src/pages/Dashboard.jsx`) was fully rewritten to:

- Use `useEffect` to fetch from `/api/applications` and `/api/applications/stats` on mount
- Show Loader component (spinner/skeleton) while fetching data
- Show Toast error notifications on fetch failures
- Support full CRUD: Create (POST), Edit (PUT), Delete (DELETE) via modal
- Live Search via `GET /api/applications/search?q=`